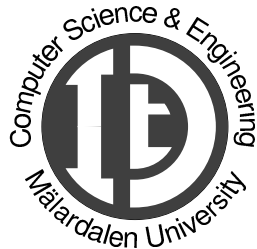


Kompendium för distanskurs CD5390 - Programkonstruktion med C

Analys/Design - och tillhörande dokumentation



Version: 1.0

Kursansvarig: Dan Levin, tfn 016-155137

Besöksadress: Rum H241, Drottninggatan 16, ESKILSTUNA

Email: dan.levin@mdh.se

Innehåll

1	Inledning	3
1.1	Analys - " <i>VAD skall programmet göra ?</i> "	3
1.2	Design - " <i>HUR skall programmet klara analys/kravspecifikation ?</i> "	3
2	Pseudokod - en beskrivningsmetod för algoritmer	4
2.1	Några exempel - med översättning till C-kod.	5
2.1.1	Funktion - returnera den största av två inparametrar	5
2.1.2	Funktion - returnera största element i en array	6
3	Slarvig analys/design - och dess följder	7

1 Inledning

Låt oss börja med att titta på de 2 *grundbultarna* i *seriös* programmering:

- Analys
- Design

1.1 Analys - “*VAD skall programmet göra ?*”

Genom analysen försöker vi specificera *programmets funktion* - dvs *VAD* programmet skall åstadkomma. Funktionen måste beskrivas *i detalj*, så att det inte råder minsta tvekan om hur det beter sig - när programmet är klart.

Analysen utmynnar i ett *dokument* - en s k *kravspecifikation*. *Kravspecifikationen* skrivs ofta på vanlig svenska (ren text). Analysen - dvs innehållet i *kravspecifikationen* - skall ge svar på bl a följande frågor:

- *Vad* förväntas programmet göra ?
- Vilka *utdata* ska presenteras ?
- Vilka *indata* behövs för att programmet skall funka ?
- Hur skall *indata levereras* till programmet ? Skall programmet få sina indata från fil/tangentbord/mus - eller annan inenhet.
- Hur skall *utdata presenteras* ? Skall resultatet skrivas på fil/skrivare/bildskärm ? Skall utdata presenteras som text/grafik.
- Vilka *begränsningar* finns det för programmet ? Hur stora tal ska klaras av ? Vilken precision skall sifferdata ha ? Hur stora datamängder skall hanteras ?

Analysen skall vara så *detaljerad och entydig*, att det inte råder någon tvekan om huruvida programmet verkligen försöker lösa rätt problem(=kundens krav) - då programmet är färdigt. Om så inte är fallet, så löser vi förmodligen inte rätt problem ...

1.2 Design - “*HUR skall programmet klara analys/kravspecifikation ?*”

När vi kommit fram till *designfasen*, så behöver vi bl a använda oss av vår erfarenhet om vad som är möjligt att utföra med ett datorprogram.

Om det är ett *större problem*, som ska lösas, så försöker vi *dela upp* det i *avgränsade delar* - *moduler*. Därefter försöker vi beskriva hur dessa delar förhåller sig till varandra. Varje delproblem(modul) kan sedan lösas - för sig.

Under designen gäller det att hitta och beskriva de *algoritmer*, som ska användas. *Vitsen med designen* är bl a att *kodningsfasen ska bli enklare* - och att vi därigenom kommer att göra färre fel vid kodningen. Detta bidrar till att programmet blir färdigt i tid.

Dokumentet, som vi skapar under designfasen, heter *systembeskrivning*. Vi skall under designen använda oss av ett hjälpmedel, som heter *pseudokod*. Under hela tiden måste vi ha *kravspecifikationen* - som utgångspunkt.

Då man vill ha *systembeskrivningen* som en förberedelse för nästa fas - *kodningsfasen* - så är det en fördel att använda sig av en *beskrivningsmetod med formella inslag* - då det härigenom blir lättare att hitta en motsvarighet i det aktuella programmeringsspråket - t ex C.

En sådan *formell beskrivningsmetod* är *pseudokod*.

2 Pseudokod - en beskrivningsmetod för algoritmer

Pseudokod är en *formell och programspråksoberoende beskrivningsmetod* för ett datorprogram. Det finns fler¹².

Kravet på *programspråksoberoende* hos pseudokoden är ganska naturligt. Denna kan ju i så fall utnyttjas av flera programmerare, som jobbar med *olika* språk - en klar fördel. Egentligen så handlar ju programmering om att lösa problem på ett strukturerat sätt i en dator - och dokumentera lösningen - oberoende av ett specifikt språk.

Språket för pseudokoden är ett naturligt språk - t ex ditt modersmål eller engelska.

Vokabulären, dvs de ord som används, är ganska fri. Detta är en klar fördel, eftersom man kan *koncentrera* sig på att hitta rätt algoritm - och slipper lägga för mycket jobb på att skriva rätt syntax. Man använder vanliga ord - ofta i imperativ (uppmanande) form - för att direkt tala om vad man vill göra. Förutom vanliga ord så *kan* vissa *datatekniska termer* förekomma i pseudokoden. Detta är dock *inget självändamål*, utan ju mer "vanligt" språk du kan använda - desto bättre.

För att beskriva de *styrstrukturer*, som finns i alla programspråk - t ex *selektion* och *iteration* - så används med fördel vissa speciella *nyckelord*. Dessa nyckelord skrives med *versaler* (se nedan) - för att ytterligare understryka deras betydelse i programmering. För en *selektion* används t ex:

En enkel *if*-konstruktion ...

På svenska:

OM *villkor*
sats1
OM-END

På engelska:

IF *condition*
expression1
IF-END

En *if-else*-konstruktion...

OM *villkor*
sats1
ANNARS
sats2
OM-ANNARS-END

IF *condition*
expression1
ELSE
expression2
IF-ELSE-END

Lägg märke till indragningarna för 'sats1' och 'sats2' ovan ! Dessa hjälper till att visa själva *selektionen* - dvs det är *antingen* 'sats1' eller 'sats2' som utföres, beroende på om '*villkor/condition*' är sant eller falskt !

För *iteration/loop* kan man skriva t ex så här:

Först en *0-slinga* - som alltid körs 0 eller fler ggr ...

På svenska:

UPPREPA MEDAN *villkor*
sats3
UPPREPA-END

På engelska:

WHILE *condition*
expression3
WHILE-END

...dvs, 'sats3' upprepas så länge som '*villkor/condition*' är sant ! Även här göres en indragning av 'sats3', för att visa vad det är, som skall upprepas. Detta bidrar till att förtydliga iterationen i pseudokoden !

¹Flödesschema

²Jackson Structured Programming

Här kommer en *1-slinga* ...

På svenska:

UPPREPA
sats3
MEDAN *villkor*

På engelska:

DO
expression3
WHILE *condition*

Här ser vi att '*villkor/condition*' kommer *efter* 'sats3' - varför 'sats3' körs *minst* en gång.

En *flervals/switch*-konstruktion kan man skriva som nästlade *if-else*-konstruktioner.

Delproblemen(=funktionerna) avgränsas också med nyckelord. *Huvudfunktionen* - där programexekveringen börjar - kan t ex skrivas så här i pseudokod:

På svenska:

HUVUDFUNKTION
Specifika ev. inparametrar
Deklarera lokala variabler
...
satser
...
HUVUDFUNKTION-END

På engelska:

MAINFUNCTION
Specify formal parameters, if any
Declare local variables
...
expressions
...
MAINFUNCTION-END

Övriga funktioner, som alltid ges ett namn av programmeraren - och kan skrivas så här:

På svenska:

FUNKTION-XYZ
Specifika ev. inparametrar
Deklarera lokala variabler
...
satser
...
Returnera ev. ett värde
FUNKTION-XYZ-END

På engelska:

FUNCTION-XYZ
Specify formal parameters, if any
Declare local variables
...
expressions
...
Return a value, if any
FUNCTION-XYZ-END

2.1 Några exempel - med översättning till C-kod.

Låt oss titta på några exempel - där jag översätter en design direkt till motsvarande C-kod.

2.1.1 Funktion - returnera den största av två inparametrar

Pseudokod:

FUNKTION-MAX
Inparametrar: tal1, tal2
Lokal variabel: maxval
OM 'tal1' > 'tal2'
Returnera 'tal1'
ANNARS
Returnera 'tal2'
OM-ANNARS-END
FUNKTION-MAX-END

Programkod - i C:

```
int max(int tal1, int tal2)
{
    int maxval;
    if (tal1 > tal2)
        return tal1;
    else
        return tal2;
}
```

Observera apostrofer runt *variabler* - som t ex 'tal1' och 'tal2' - i pseudokoden. Motivet till detta är att man lätt skall hitta variabelnamn i pseudokoden. Vi kan t ex ha valt variabelnamn, som "flyter ihop" med övrig text i pseudokoden och som därför blir svåra att upptäcka.

2.1.2 Funktion - returnera största element i en array

Pseudokod:

FUNKTION-ARRMAX

Inparametrar: arr, len

Lokala variabler: i, max

Sätt 'i'=1. Sätt 'max'='arr[0]'.

UPPREPA MEDAN 'i' < 'len'

OM 'arr[i]' > 'max'

Sätt 'max'='arr[i]'.

OM-END

Öka 'i' med 1.

UPPREPA-END

Returnera 'max'.

FUNKTION-ARRMAX-END

Programkod - i C:

```
int maxarr(int arr[], int len)
```

```
{
```

```
int i, max;
```

```
i=1;max=arr[0];
```

```
while (i<len)
```

```
{
```

```
if (arr[i]>max)
```

```
max=arr[i];
```

```
i++;
```

```
}
```

```
return max;
```

```
}
```

...och här kommer en *huvudfunktion*, som anropar funktionen ARRMAX ovan:

Pseudokod:

HUVUDFUNKTION

Inparametrar: inga

Lokal variabel: myarr, maxval.

Läs in 10 heltal i 'myarr'.

Sätt 'maxval'=ARRMAX('myarr', 10)

Skriv ut 'maxval'.

HUVUDFUNKTION-END

Programkod - i C:

```
int main(void)
```

```
{
```

```
int myarr[10], maxval, i;
```

```
for (i=0; i<10; i++)
```

```
{
```

```
printf("\nMata in ett heltal: ");
```

```
scanf("%d", &myarr[i]);
```

```
}
```

```
maxval=maxarr(myarr, 10);
```

```
printf("\n\nMaxvärde: %d", maxval);
```

```
return 0;
```

```
}
```

I sista exemplet ser du hur vi kan *förenkla* pseudokoden för *huvudfunktionen*, eftersom det är funktionen ARRMAX, som är den *viktiga* här. Funktioner, som bara *håller ihop* andra funktioner, kan man med fördel förenkla pseudokodmässigt. Det är *arbetsfunktionerna* - dvs de som gör själva jobbet via någon algoritm, som man oftast beskriver i detalj i pseudokoden.

Nu är det dags för dig själv att börja skriva pseudokod på allvar.

3 Slarvig analys/design - och dess följder

- Kunden kommer på - efter att ha använt programmet några år - att det skulle behövas lite mer finesser i programmet. Detta kan knappast införas i det befintliga programmet - vare sig av programmeraren eller av någon annan - eftersom dokumentation saknas/är bristfällig.
- Efter något år har man själv antagligen glömt det mesta.
- Programmet fungerar bra för det mesta - men inte alltid. Kunden klagar först efter en tid. Då kan det vara mycket svårt att få någon rätsida på programmet, då man saknar stöd av analys/design. Det är t.ex. inte troligt att någon annan kan eller vill ta över arbetet med att förändra programmet.
- Programmet fungerar perfekt - men det löser inte det problem, som *kunden* ville ha löst. Om man inte följer en *konsekvent* modell, så är det lätt hänt att man låter egna spontana infall ta över kommandot - dvs delvis glömmar kundens krav.
- Programmet blir aldrig klart! Programmet blir ett lapptäcke av fixar. Eftersom man inte har en struktur att falla tillbaks på, så försöker man kringgå upptäckta brister genom klistra dit nya variabler och funktioner lite på måfå $\Rightarrow$ Katastrof - tidsmässigt och ekonomiskt !

Slutsats: Jag hoppas alltså, att du numera är helt *övertygad* om förträffligheten med *väl genomförd analys/design* - i *seriöst* programmeringsarbete ...